Kiểm Soát Chất Lượng & Đánh Giá Tự Động Cho AI Agent (LLM)

Khung Đánh Giá Đa Tầng (Multi-Layered Evaluation), Đo Lường Hành Vi Thực Tế & A/B Testing Cho Hệ Thống Travel Booking Assistant

> **Dự án:** Travel Assistant System (LangGraph · ASP.NET Core · vLLM · 9Router)
>
> **Chủ đề:** Khắc phục hiện tượng Plausible Hallucination, xây dựng bộ đo lường tất định & kiểm nghiệm A/B Testing
>
> **Ngày lập báo cáo:** 02/11/2026

1. Đặt Vấn Đề: Căn Bệnh "Plausible Hallucination"

Khi triển khai các mô hình ngôn ngữ lớn (LLM) vào các hệ thống hỗ trợ nghiệp vụ thực tế (như đặt vé máy bay, khách sạn, lập lịch trình), rủi ro lớn nhất không phải là mô hình không trả lời được, mà là **mô hình trả lời rất trôi chảy, tự tin nhưng nội dung không chính xác hoặc không mang lại giá trị cho người dùng**.

> ⚠️ **Hai Rủi Ro Lớn Nhất Trong Môi Trường Production:**
> 1. **Dữ liệu ảo hóa (Inventory Hallucination):** Agent hứa hẹn các tiện ích khách sạn không có thực, báo sai khoảng giá, hoặc đề xuất các chuyến bay có thời gian transit vô lý.
> 2. **Thiếu tính hành động (Low Task Actionability):** Agent trả lời theo kiểu bách khoa toàn thư dài dòng, thiếu các nút liên kết đặt phòng cụ thể, không hỗ trợ khách chốt đơn được trên hệ thống.

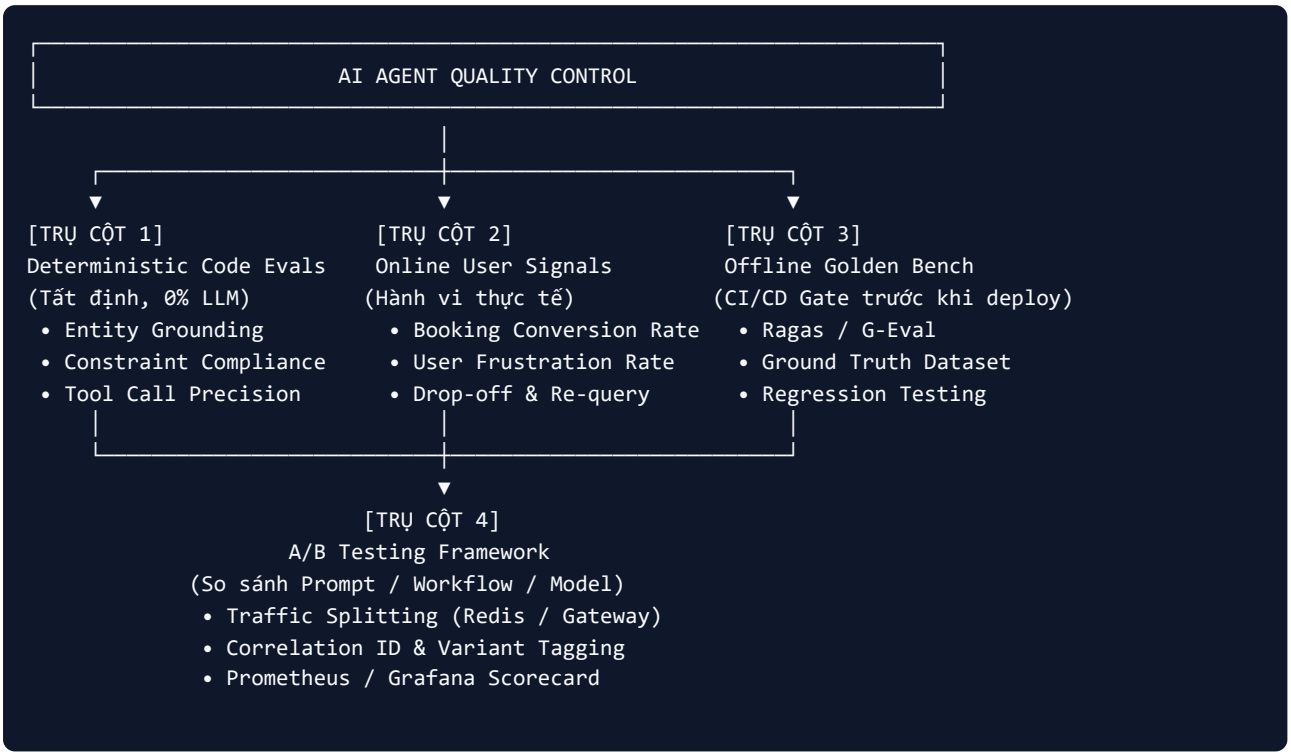
Giới Hạn Của Cơ Chế Tự Đánh Giá (In-Graph LLM Evaluation) Hiện Tại

Hệ thống hiện tại đã tích hợp các node `fact_checker` và `evaluator_agent` trong LangGraph. Tuy nhiên, việc phụ thuộc hoàn toàn vào LLM-as-a-judge trong runtime có các điểm yếu cốt tử:

- **Circular Blind Spots (Điểm mù chéo):** Node Evaluator và Node Executor thường dùng chung một họ mô hình (hoặc chung manifest tư duy). Mô hình có xu hướng đồng tình với các giả định của chính nó (Sycophancy bias).
- **Độ trễ và Chi phí (Latency & Token Tax):** Phải tốn thêm từ 1 đến 2 lần gọi LLM suy luận chỉ để tự đánh giá, làm tăng TTFT (Time To First Token) và kéo dài tổng thời gian phản hồi thêm 1.5 – 3 giây.
- **Thiếu dữ liệu thực tế (Zero Ground Truth):** LLM không thể tự biết được khách hàng sau câu trả lời đó có ấn nút "Đặt Phòng" hay tức giận tắt ứng dụng.

2. Kiến Trúc Kiểm Soát Chất Lượng Đa Tầng (4 Trụ Cột)

Để kiểm soát chất lượng một cách toàn diện, chúng tôi thiết kế khung đánh giá kết hợp giữa **Quy tắc mã nguồn tất định**, **Hành vi thực tế của khách hàng**, **Bộ kiểm thử ngoại tuyến tự động** và **Khung thử nghiệm A/B Testing**.



Chi Tiết Các Trụ Cột Đánh Giá

📌 Trụ Cột 1: Đánh Giá Tất Định Bằng Mã Nguồn (Deterministic Verification - 0% LLM)

Được thực thi bằng logic code C#/Python thuần ngay sau khi Agent kết thúc lượt sinh token. Chi phí 0 token, thời gian xử lý < 5ms.

Chỉ số (Metric)	Cách Đo & Công Thức	Ý Nghĩa Nghiệp Vụ
Entity Grounding Score	`Khớp thực thể trích xuất / Thực thể thực tế trong DB`	Xác minh khách sạn, chuyến bay, giá phòng nêu ra có tồn tại thật trong cơ sở dữ liệu hay không.
Constraint Compliance Rate	`Kiểm tra ngày nhận < ngày trả; Giá ≤ Ngân sách user`	Đo lường mức độ tuân thủ các điều kiện tiên quyết mà người dùng đã chỉ định trong prompt.
Wasted Tool Rate	`Số Tool gọi ra nhưng kết quả bị bỏ qua / Tổng số Tool`	Phát hiện tình trạng Agent gọi tool lãng phí, làm chậm tốc độ stream của người dùng.

📌 Trụ Cột 2: Tín Hiệu Hành Vi Thực Tế Của Người Dùng (Online Behavioral Metrics)

Tín hiệu hành vi phản ánh chính xác nhất mức độ hữu ích của câu trả lời, không cần người dùng phải bấm sao đánh giá.

Chỉ số	Cơ Chế Thu Thập	Ngưỡng Cảnh Báo
Task Conversion Rate (TCR)	Gắn `session_id` với sự kiện user click *"Đặt phòng"* hoặc chuyển tới trang thanh toán Stripe.	Nếu TCR < 15% đối với các câu hỏi có ý định đặt phòng $\rightarrow$ Cảnh báo Prompt tư vấn kém hiệu quả.
User Frustration Rate	Phát hiện các từ khóa tiêu cực (*"sai rồi"*, *"không phải"*, *"tìm lại"*) hoặc nhập lại cùng câu hỏi trong vòng 60 giây.	Tỷ lệ hỏi lại > 12% phản ánh Agent đang trả lời không trúng ý người dùng.
Turn-to-Resolution (TTR)	Số lượng vòng hội thoại cần thiết để khách hàng chọn được một dịch vụ cụ thể.	TTR tối ưu: 2 – 4 turns. Nếu kéo dài > 7 turns phản ánh Agent vòng vo.

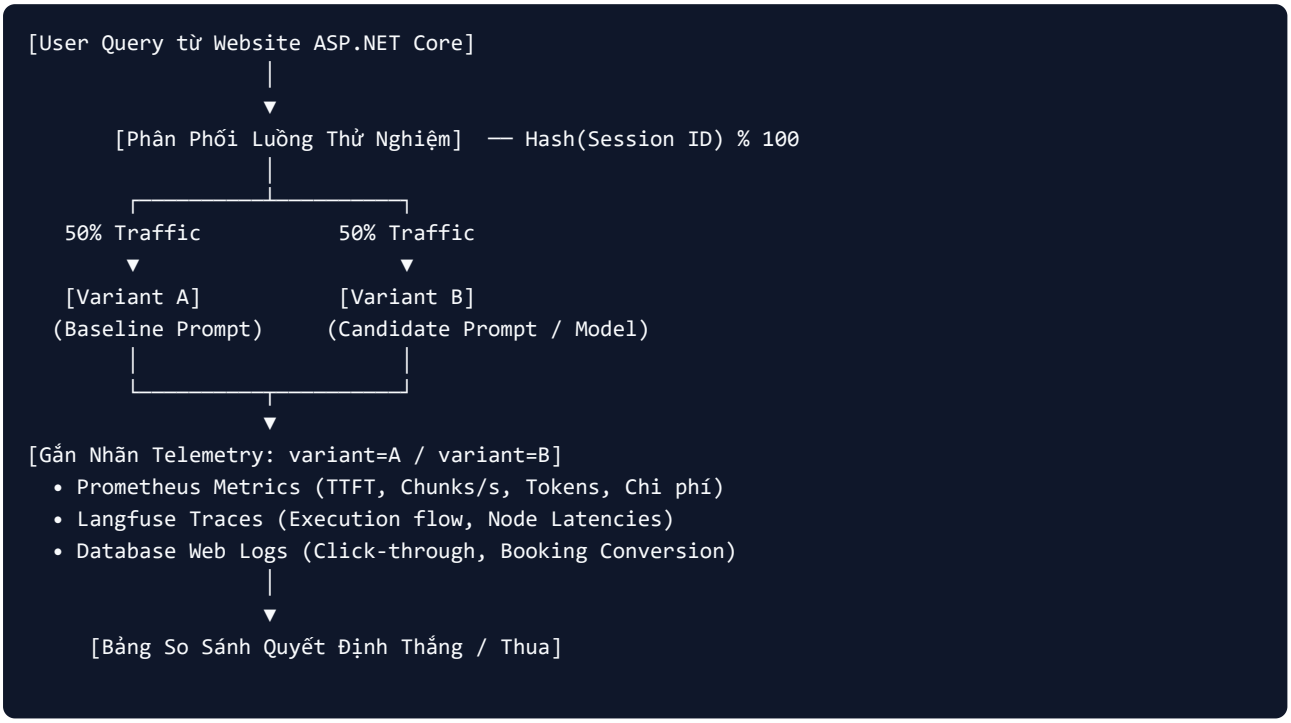
📌 Trụ Cột 3: Bộ Kiểm Thử Ngoại Tuyến & CI/CD Gate (Offline Benchmark Suite)

Bộ đánh giá bắt buộc phải chạy tự động trước khi một bản cập nhật prompt, model hay node LangGraph được đưa lên production.

- **Golden Dataset (100–200 câu hỏi chuẩn):** Phân chia cân bằng giữa các câu hỏi dễ (FAQ), trung bình (lập lịch trình), phức tạp (ngân sách hạn hẹp), và bẫy ngoại biên (out-of-domain, injection).
- **Chấm điểm chuẩn quốc tế (Ragas / DeepEval):**
  - `Faithfulness (Độ trung thực):` Tỷ lệ các khẳng định được chứng minh trực tiếp từ dữ liệu trả về của Tool.
  - `Answer Relevance (Độ liên quan):` Mức độ bám sát trực diện vào mục tiêu người dùng yêu cầu.
  - `Language Anti-Drift:` Đảm bảo tỷ lệ chữ tiếng Trung trong phản hồi tiếng Việt/Anh bằng 0%.
- **Quality Gate:** Tự động chặn lệnh Deploy nếu điểm `Faithfulness < 0.90` hoặc tỷ lệ tuân thủ điều kiện `Constraint < 95%`.

3. Thiết Kế Hệ Thống A/B Testing Cho AI Agent

Để kiểm nghiệm xem Prompt mới, Mô hình mới (ví dụ so sánh **9Router GPT-4o** vs **vLLM Qwen2.5-AWQ**), hoặc Kịch bản phân rã mới có tốt hơn hay không, hệ thống cần được trang bị luồng phân phối thử nghiệm chuẩn:



Ma Trận Tiêu Chí Đánh Giá Quyết Định (A/B Test Scorecard)

Một ứng viên (Variant B) chỉ được phép thay thế phiên bản cũ (Variant A) khi thỏa mãn toàn bộ các điều kiện trong ma trận sau:

Khía Cạnh Đánh Giá	Chỉ Số Đo Lường (Metrics)	Điều Kiện Chiến Thắng Của Variant B
Chất lượng nghiệp vụ	Tỷ lệ chuyển đổi đặt phòng (Conversion Rate)	**Tăng ≥ 5%** so với Variant A (p-value < 0.05).
Độ chính xác dữ liệu	Entity Grounding & Constraint Compliance	**≥ 98%**, không được có bất kỳ ca suy giảm độ chính xác nào.
Độ trễ phản hồi	P95 Time to First Token (TTFT)	Không được tăng quá **10%** so với bản cũ.
Chi phí vận hành	Chi phí ước tính / 1000 lượt chat ($ USD)	**Giảm hoặc tương đương** chi phí của Variant A.
Trải nghiệm tương tác	User Frustration Rate (Hỏi lại / Thoát sớm)	**Giảm ≥ 10%** so với Variant A.

4. Lộ Trình Triển Khai Kỹ Thuật Cho Dự Án Booking

🚀 **Kế Hoạch 3 Bước Tận Dụng Hạ Tầng Có Sẵn:**

1. **Bước 1 — Post-Session Audit Worker (Tận dụng RabbitMQ hiện có):** Hiện tại sau mỗi phiên chat, hệ thống đã publish tin nhắn vào RabbitMQ cho `MemoryWorker`. Chúng ta có thể mở rộng worker này để chạy thêm bước kiểm tra đối soát chéo (Cross-Audit) giữa văn bản sinh ra với cơ sở dữ liệu khách sạn, tính `Grounding Score` và đẩy trực tiếp thành Prometheus metric mới `agent_conversation_quality_score`.
2. **Bước 2 — Telemetry & Conversion Tracking Tại ASP.NET Core:** Trong SignalR `ChatHub.cs` và `BookingController.cs`, khi người dùng bấm tạo đơn đặt phòng hoặc thanh toán Stripe thành công, tự động gửi kèm `session_id` để lưu vết chuyển đổi thành công.
3. **Bước 3 — Xây dựng Bộ Evals Tự Động Trong Thư Mục `agent/tests/evals/`:** Tận dụng ngân hàng 60 câu hỏi có sẵn tại `agent/load_tests/questions_db.json`, viết kịch bản Python tự động chấm điểm độ trung thực và tính tuân thủ của mô hình trước mỗi lần merge code.